



南开大学
Nankai University

《计算机网络》实验报告

(2025~2026 学年第一学期)

实验名称：基于套接字的网络程序设计

学 院：软件学院

姓 名：李宗杭

学 号：2311451

指导老师：辜文蔚

2025 年 12 月 27 日

目录

实验 1 套接字基础与 UDP 通信	1
1 实验目的	1
2 实验条件	1
3 实验内容及原理	1
3.1 实验原理	1
3.2 实验步骤	2
3.3 代码说明	2
3.4 实验结果	5
4 问题与体会	8
实验 2 TCP 通信与 Web 服务器	9
1 实验目的	9
2 实验条件	9
3 实验内容及原理	9
3.1 实验原理	9
3.2 实验步骤	10
3.3 代码说明	10
3.4 实验结果	13
4 问题与体会	14
实验 3 SMTP 客户端实现	15
1 实验目的	15
2 实验条件	15
3 实验内容及原理	15
3.1 实验原理	15
3.2 实验步骤	15
3.3 代码说明	16
3.4 实验结果	16
4 问题与体会	17

实验 1 套接字基础与 UDP 通信

1 实验目的

熟悉基于 Python 进行 UDP 套接字编程的基础知识，掌握使用 UDP 套接字发送和接收数据包，以及设置正确的套接字超时，了解 Ping 应用程序的基本概念，并理解其在简单判断网络状态，例如计算数据包丢失率等统计数据方面的意义。

2 实验条件

装有 python 环境的电脑两台；
局域网环境。

3 实验内容及原理

3.1 实验原理

基于 UDP 的无连接客户/服务器在 Python 实现中的工作流程如下：

1. 首先在服务器端通过调用 `socket()` 创建套接字来启动一个服务器。
2. 服务器调用 `bind()` 指定服务器的套接字地址，然后调用 `recvfrom()` 等待接收数据。
3. 在客户端调用 `socket()` 创建套接字，然后调用 `sendto()` 向服务器发送数据。
4. 服务器接收到客户端发来的数据后，调用 `sendto()` 向客户发送应答数据。
5. 客户调用 `recvfrom()` 接收服务器发来的应答数据。
6. 一旦数据传输结束，服务器和客户通过调用 `close()` 来关闭套接字。

3.2 实验步骤

1. 运行服务端，循环接收客户端消息。
2. 客户端使用 UDP 发送 ping 消息。
3. 如果服务器在 1 秒内响应，则打印该响应消息并计算每个数据包的往返时间 RTT(以秒为单位)；否则，打印“请求超时”。

3.3 代码说明

UDP 服务器：接收客户端发送的消息、将消息转换为大写后回传客户端、模拟 40% 概率的数据包丢失、收到“exit”指令时正常关闭服务器。

1. 创建并绑定 UDP 套接字：使用 `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` 创建 UDP 类型套接字（SOCK_DGRAM 是 UDP 协议的标识），通过 `bind(('', 10000))` 绑定到本机所有可用网络接口的 10000 端口，监听客户端消息。
2. 循环接收客户端消息：使用 `recvfrom(1024)` 接收客户端消息，返回消息内容和客户端地址，UDP 无连接，需记录客户端地址以回传数据，并将字节流消息解码为 UTF-8 字符串打印。
3. 退出逻辑处理：判断接收的消息是否为“exit”，若是则打印关闭信息并跳出循环，终止服务器运行。
4. 消息处理与丢包模拟：使用 `random` 模拟 40% 丢包概率，满足条件时打印“数据包丢失”并跳过回传，模拟 UDP 协议无可靠传输保障的特性；未丢包时，通过 `sendto(message, address)` 将大写消息回传给对应的客户端。

```
1  import socket
2  import random
3
4  # 创建UDP服务器套接字, AF_INET表示使用IPv4, SOCK_DGRAM表示使用UDP协议
5  serverSocket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6  # 绑定服务器地址和端口号, 第一个参数为空字符串表示监听所有可用的接口
7  serverSocket.bind(('', 10000))
8  print("UDP服务器已启动, 等待接收消息...\n")
9
10 # 循环接收和处理客户端消息
11 while True:
12     # 接收客户端消息, 缓冲区大小为1024字节
13     message, address = serverSocket.recvfrom(1024)
14     print("接收的消息: {}".format(message.decode("utf-8")))
15     print("接收的地址: {}\n".format(address))
16     # 如果接收到的消息是"exit", 则退出循环, 关闭服务器
17     if message.decode("utf-8") == "exit":
18         print("服务器已关闭。")
19         serverSocket.close()
20         break
21     # 简单的消息处理: 将消息转换为大写
22     message = message.upper()
23     # 模拟数据包丢失的情况, 40%的概率丢失数据包
24     if random.randint(0, 10) < 4:
25         print("数据包丢失\n")
26         continue
27     # 发送处理后的消息回客户端
28     serverSocket.sendto(message, address)
```

图 1.1: udp_server

UDP 客户端：向 UDP 服务器发送 10 次“ping”请求，统计往返时延（RTT）和丢包率，包括最小 RTT、最大 RTT、平均 RTT，最后发送“exit”指令关闭服务器。

1. 创建 UDP 套接字并设置超时：使用 `socket.socket()` 创建 UDP 套接字，通过 `settimeout(1)` 设置 1 秒超时时间，超过 1 秒未收到响应则视为丢包。
2. 循环发送 Ping 请求：每次用 `time.time()` 记录发送开始时间 `t1`，通过 `sendto()` 将“ping”编码为 UTF-8 字节流，发送到指定服务器地址和 10000 端口，用 `try-except` 捕获超时异常：超时则打印超时信息并累加丢包数 `count`，跳过后续逻辑，未超时则用 `recvfrom()` 接收服务器响应，记录接收完成时间 `t2`。
3. RTT 统计计算：初始化标记 `start`，第一次收到响应时初始化最小 RTT（`mintime`）和最大 RTT（`maxtime`），后续响应中，对比当前 RTT（`t2-t1`），更新最小/最大 RTT，并累加总 RTT（`totaltime`）。
4. 结束处理：循环结束后发送“exit”指令关闭服务器，关闭客户端套接字，最后计算并打

印平均 RTT 和丢包率。

```

1  import socket
2  import time
3
4  port = 10000
5  # address = "127.0.0.1"
6  address = "192.168.31.36"
7  count = 0 # 丢包数
8  start = 0 # 标记是否为第一次接收消息
9  totaltime = 0 # 总往返时延RTT
10
11 # 创建UDP客户端套接字
12 clientSocket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
13 # 设置超时时间（往返时延RTT）为1秒
14 clientSocket.settimeout(1)
15
16 # 发送10次ping消息并计算RTT
17 for i in range(10):
18     print("test_" + str(i) + "_RRT: ", end = '')
19     t1 = time.time() # 记录发送时间
20     # 发送ping消息到服务器，Socket传输的是字节流，因此需要将字符串编码为字节
21     clientSocket.sendto("ping".encode("utf-8"), (address, port))
22     try:
23         # 接收服务器的响应消息，缓冲区大小为1024字节
24         message, recv_address = clientSocket.recvfrom(1024)
25     except:
26         # 超时未收到响应，视为丢包
27         print("out of time!!!")
28         count = count + 1
29         continue
30     t2 = time.time() # 记录接收时间
31
32     # 计算往返时延RTT并更新最小、最大和平均RTT
33     if (start == 0): # 第一次接收消息，初始化最小和最大RTT
34         mintime = (t2 - t1)
35         maxtime = (t2 - t1)
36         start = 1
37     elif ((t2 - t1) < mintime):
38         mintime = (t2 - t1)
39     elif ((t2 - t1) > maxtime):
40         maxtime = (t2 - t1)
41     totaltime = totaltime + (t2 - t1)
42     print('%.15f' % (t2 - t1))
43
44 # 发送退出消息给服务器
45 clientSocket.sendto("exit".encode("utf-8"), (address, port))
46 # 关闭客户端套接字
47 clientSocket.close()
48
49 # 输出RTT统计信息和丢包率
50 print('min_RRT:' + str(mintime))
51 print('max_RRT:' + str(maxtime))
52 print('average_RRT:' + str(totaltime / (10 - count)))
53 print('packet loss rate:' + str(count / 10))
54

```

图 1.2: udp_client

3.4 实验结果

首先在本地进行测试，并随机模拟丢包率。从测试结果中可以看到服务端与客户端能成功进行通信，有 60% 丢包率，并且可以正确的计算各类 RTT，因为网络状态很好，延迟极低，且计时精度有限，有可能 RTT 是 0。

```
(llm) PS D:\VSCodeProject\homework\ComputerNet> python udp_server.py
UDP服务器已启动，等待接收消息...

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

数据包丢失

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

数据包丢失

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

数据包丢失

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

数据包丢失

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

数据包丢失

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

数据包丢失

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

接收的消息: ping
接收的地址: ('127.0.0.1', 56188)

接收的消息: exit
接收的地址: ('127.0.0.1', 56188)

服务器已关闭。
```

图 1.3: udp_server_local

```
(llm) PS D:\VSCodeProject\homework\ComputerNet> python udp_client.py
test_0_RRT: out of time!!!
test_1_RRT: 0.002181291580200
test_2_RRT: out of time!!!
test_3_RRT: out of time!!!
test_4_RRT: out of time!!!
test_5_RRT: out of time!!!
test_6_RRT: out of time!!!
test_7_RRT: 0.0000000000000000
test_8_RRT: 0.0000000000000000
test_9_RRT: 0.0000000000000000
min_RRT:0.0
max_RRT:0.0021812915802001953
average_RRT:0.0005453228950500488
packet loss rate:0.6
```

图 1.4: udp_client_local

在两台主机进行测试时，注释丢包模拟，测试真实网络。可以看到丢包率为 0，且大部分 RTT 也为 0，也是因为两台设备在同一局域网内，网络状态很好，延迟极低。

```
(test) PS D:\Lizonghang\ComputerNet> python udp_server.py
UDP服务器已启动, 等待接收消息...

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'ping'
接收的地址: ('192.168.31.197', 49563)

接收的消息: b'exit'
接收的地址: ('192.168.31.197', 49563)

服务器已关闭。
```

图 1.5: udp_server_diff

```
(llm) PS D:\VSCoDeProject\homework\ComputerNet> python udp_client.py
test_0_RRT: 0.0000000000000000
test_1_RRT: 0.010003089904785
test_2_RRT: 0.0000000000000000
test_3_RRT: 0.0000000000000000
test_4_RRT: 0.0000000000000000
test_5_RRT: 0.0000000000000000
test_6_RRT: 0.0000000000000000
test_7_RRT: 0.015690803527832
test_8_RRT: 0.0000000000000000
test_9_RRT: 0.0000000000000000
min_RRT:0.0
max_RRT:0.01569080352783203
average_RRT:0.0025693893432617186
packet loss rate:0.0
```

图 1.6: udp_client_diff

4 问题与体会

报告中所示代码对老师提供的代码进行了优化，将地址端口设为变量，便于修改，并新增服务端退出功能，并更正了 RTT 的计算逻辑（原代码对所有结果都除以 2，不知道是何意味），另外添加了更多的打印信息用来观察实验过程。

对代码的修改还导致测试时出现了一个问题，经检查发现是在客户端中，由于使用 `address` 字符串变量作为地址，但 `recvfrom()` 方法又用 `address` 接收，并且接收的是元组，修改了 `address` 的类型，就导致下一次循环中再用 `address` 发消息会出现错误。只需将 `recvfrom()` 的接受变量改为 `recv_address`，区分两个变量便可。

通过 UDP Ping 程序实验，我直观感受了 UDP 无连接、不可靠的特性，熟练掌握了 Python UDP 套接字的创建、收发及超时设置。计算 RTT 和丢包率的过程，让我理解了如何在应用层应对网络传输的不可靠性，也清晰了不同传输层协议的选型逻辑。

实验 2 TCP 通信与 Web 服务器

1 实验目的

熟悉基于 Python 进行 TCP 套接字编程的基础知识，理解 HTTP 报文格式，能基于 Python 编写一个可以一次响应一个 HTTP 请求，并返回静态文件的简单 Web 服务器。

2 实验条件

装有 python 环境的电脑两台；
局域网环境。

3 实验内容及原理

3.1 实验原理

基于 TCP 的面向客户端/服务器在 Python 实现中的的工作流程如下：

1. 首先在服务器端通过调用 `socket()` 创建套接字来启动一个服务器。
2. 服务器调用 `bind()` 绑定指定服务器的套接字地址（IP 地址 + 端口号）。
3. 服务器调用 `listen()` 做好侦听准备，同时规定好请求队列的长度。
4. 服务器进入阻塞状态，等待客户的连接请求。
5. 服务器通过 `accept()` 来接收连接请求，并获得客户的 socket 地址。
6. 在客户端通过调用 `socket()` 创建套接字。
7. 客户端调用 `connect()` 和服务器建立连接。
8. 连接建立成功后，客户端和服务器之间通过调用 `read()` 和 `write()` 来接收和发送数据。
9. 数据传输结束后，服务器和客户各自通过调用 `close()` 关闭套接字。

3.2 实验步骤

1. 服务器收到请求时创建一个 TCP 套接字。
2. 通过这个 TCP 套接字接收 HTTP 请求
3. 解析 HTTP 请求并在操作系统中确定客户端所请求的特定文件。
4. 从服务器的文件系统读取客户端请求的文件。
5. 当被请求文件存在时, 创建一个由被请求的文件组成的“请求成功”HTTP 响应报文。
6. 当被请求文件不存在时, 创建“请求目标不存在”HTTP 响应报文。
7. 通过 TCP 连接将响应报文发回客户端。

3.3 代码说明

TCP 服务器: 能够处理客户端的 HTTP GET 请求, 返回指定的 HTML 静态文件 (200 OK 响应), 若文件不存在则返回 404 错误, 基于 TCP 协议保证可靠传输。

1. 创建并配置 TCP 服务器套接字: 使用 `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` 创建 TCP 类型套接字 (SOCK_STREAM 是 TCP 协议的标识), 用 `setsockopt()` 设置端口复用, 避免服务器重启时端口占用问题, 通过 `bind(('', 6121))` 绑定端口, `listen(10)` 设置最大挂起连接数为 10, 进入监听状态。
2. 多线程处理并发连接: 主线程通过循环调用 `accept()` 阻塞等待客户端连接, 获取与客户端的通信套接字和客户端地址, 每收到一个新连接, 创建一个子线程, 将通信套接字和客户端地址传入 `handle_client` 函数, 设置 `daemon=True` 保证主线程退出时子线程自动退出, 启动子线程处理该客户端请求, 实现多客户端并发处理。
3. 单个客户端请求处理: 接收并解析 HTTP 请求: 通过 `recv(1024)` 接收客户端的 HTTP GET 请求字节流, 解码为字符串后, 通过 `split()` 分割请求行, 提取请求的文件名。正常响应 (200 OK): 若文件存在, 读取文件内容, 构造符合 HTTP 1.1 规范的响应报文 (状态行、响应头、响应体), 编码为字节流后通过 `send()` 发送给客户端; 异常响应 (404 Not Found): 捕获 `IOError` 异常, 读取 `error.html` 错误页面内容, 构造 404 响应报文并发送。最终关闭与该客户端的通信套接字, 释放连接资源。

4. 程序退出：捕获 KeyboardInterrupt (Ctrl+C) 信号，打印退出信息并关闭服务器套接字。

```

1  import socket
2  import time
3  import threading
4
5  # 单个客户端请求处理函数
6  def handle_client(connectionSocket, address):
7      print(f"[新连接] 客户端{address}已连接")
8      try:
9          # 读取客户端发送的HTTP请求消息，解码为字符串
10         message = connectionSocket.recv(1024).decode()
11         # 防止空请求（客户端异常关闭）
12         if not message:
13             connectionSocket.close()
14             print(f"[连接关闭] 客户端{address}请求为空，已断开")
15             return
16
17         # 提取请求的文件名
18         filename = message.split()[1]
19         # 打开请求的文件
20         with open(filename[1:], "r", encoding="utf-8") as f:
21             content = f.read()
22
23         # 构造HTTP 200响应消息
24         state_row = "HTTP/1.1 200 OK\r\n"
25         response_headers = (
26             f"Connection: close\r\n"
27             f>Date: {time.strftime('%a, %d %b %Y %H:%M:%S GMT', time.gmtime())}\r\n"
28             f"Server: Apache/1.3.0 (Windows)\r\n"
29             f>Last-Modified: Mon, 28 Nov 2022 00:00:00 GMT\r\n"
30             f"Content-Length: {len(content.encode('utf-8'))}\r\n" # 按字节数计算长度
31             f"Content-Type: text/html; charset=utf-8\r\n\r\n" # 规范的Content-Type
32         )
33         outputdata = state_row + response_headers + content
34
35         # 发送响应给客户端
36         connectionSocket.send(outputdata.encode("utf-8"))
37         print(f"[响应完成] 已处理客户端{address}的请求")
38     except IOError:
39         # 处理文件未找到异常
40         print(f"[文件缺失] 客户端{address}请求的文件不存在")
41         with open("error.html", "r", encoding="utf-8") as f:
42             content = f.read()
43         state_row = "HTTP/1.1 404 Not Found\r\n"
44         response_headers = (
45             f"Connection: close\r\n"
46             f>Date: {time.strftime('%a, %d %b %Y %H:%M:%S GMT', time.gmtime())}\r\n"
47             f"Server: Apache/1.3.0 (Windows)\r\n"
48             f"Content-Length: {len(content.encode('utf-8'))}\r\n"
49             f"Content-Type: text/html; charset=utf-8\r\n\r\n"
50         )
51         outputdata = state_row + response_headers + content
52         connectionSocket.send(outputdata.encode("utf-8"))
53
54     finally:
55         # 关闭客户端连接
56         connectionSocket.close()
57         print(f"[连接关闭] 客户端{address}的连接已断开")
58 
```

图 2.1: tcp_server1

```

59 # 启动服务器并监听连接
60 def main():
61     # 创建TCP服务器套接字
62     serverSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
63     serverPort = 6121
64     # 设置端口复用
65     serverSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
66     # 绑定服务器地址和端口号
67     serverSocket.bind(("", serverPort))
68     # 监听连接请求, 最大挂起连接数为10
69     serverSocket.listen(10)
70     print(f"TCP服务器已启动, 监听端口{serverPort}, 等待客户端连接...")
71
72     try:
73         # 循环接收客户端连接
74         while True:
75             # 接受客户端连接请求
76             connectionSocket, address = serverSocket.accept()
77             # 创建子线程处理当前客户端请求
78             # daemon=True: 主线程退出时子线程自动退出
79             client_thread = threading.Thread(
80                 target=handle_client, args=(connectionSocket, address), daemon=True
81             )
82             # 启动子线程
83             client_thread.start()
84     except KeyboardInterrupt:
85         # 捕获Ctrl+C
86         print("\n[服务器停止] 接收到退出信号, 关闭服务器...")
87     finally:
88         # 关闭服务器套接字
89         serverSocket.close()
90         print("[服务器关闭] TCP服务器已停止运行")
91
92 if __name__ == "__main__":
93     main()

```

图 2.2: tcp_server2

TCP 客户端：向 HTTP 服务器发送 HTTP GET 请求，获取指定 HTML 文件，统计请求往返时延（RTT），并将响应的 HTML 内容保存到本地文件。

1. 创建 TCP 套接字并连接服务器：使用 `socket.socket()` 创建 TCP 套接字，通过 `connect((address,port))` 与服务器的指定端口建立 TCP 连接（TCP 是面向连接的协议，必须先建立连接再通信）。
2. 构造并发送 HTTP GET 请求：拼接符合 HTTP 1.1 规范的 GET 请求报文（请求行、请求头），将请求报文编码为 UTF-8 字节流，通过 `send()` 发送给服务器。
3. 统计 RTT 并接收响应：使用 `time.perf_counter()` 记录请求发送前后的时间，计算往返时延 RTT；通过 `recv(1024)` 接收服务器返回的响应报文，解码为字符串并打印。
4. 保存响应内容到本地：将响应报文按 `\r\n` 分割，提取响应体，写入本地 `localHtml.html` 文件，完成文件保存。
5. 结束处理：关闭客户端套接字，断开与服务器的 TCP 连接。

```

1 import time
2 import socket
3
4 address = "127.0.0.1"
5 # address = "192.168.31.36"
6 port = 6121
7
8 # 创建TCP客户端套接字
9 clientsocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 # 连接服务器
11 clientsocket.connect((address, port))
12
13 # 构造HTTP请求消息
14 fetch_file = "/myfile.html" # 请求的文件名
15 requestRow = "Get " + fetch_file + " HTTP/1.1\r\n"
16 firstRow = "Host:localhost\r\nUser-agent:Microsoft Edge/100.0.1185.36\r\nConnection:keep-alive\r\nAccept-language:ch\r\n\r\n"
17 requestMessages = requestRow + firstRow
18
19 start = time.perf_counter() # 开始计时
20 # 发送HTTP请求消息给服务器
21 clientsocket.send(requestMessages.encode())
22 # 接收服务器的响应消息
23 responseMessage = clientsocket.recv(1024)
24 end = time.perf_counter() # 结束计时
25
26 print("RTT:", end - start, "s")
27 print("响应报文: \n", responseMessage.decode("utf-8"), sep = "")
28
29 # 将响应内容写入本地文件
30 f = open('localHtml.html', 'w')
31 message = responseMessage.decode()
32 message = message.split("\r\n")
33 content = message[-1]
34 f.write(content)
35 f.close()
36
37 # 关闭客户端套接字
38 clientsocket.close()

```

图 2.3: tcp_client

3.4 实验结果

本地测试，服务端与客户端能正常建立连接并完成通信全流程。服务端监听客户端连接，接收并处理客户端请求，最后可以正确的关闭连接。客户端也可以正确解析响应报文，并计算 RTT 和数据长度。

```

(C:\m) PS D:\VSCodeProject\homework\ComputerNet> python tcp_server.py
TCP服务器已启动, 监听端口6121, 等待客户端连接...
[新连接] 客户端('127.0.0.1', 52516)已连接
[响应完成] 已处理客户端('127.0.0.1', 52516)的请求
[连接关闭] 客户端('127.0.0.1', 52516)的连接已断开

```

图 2.4: tcp_server_local

```

(C:\m) PS D:\VSCodeProject\homework\ComputerNet> python tcp_client.py
RTT: 0.0008996000979095697 s
响应报文:
HTTP/1.1 200 OK
Connection: close
Date: Sat, 27 Dec 2025 19:12:05 GMT
Server: Apache/1.3.0 (Windows)
Last-Modified: Mon, 28 Nov 2022 00:00:00 GMT
Content-Length: 20
Content-Type: text/html; charset=utf-8

request successfully

```

图 2.5: tcp_client_local

双主机测试也能正常运行。

```
(test) PS D:\Lizonghang\ComputerNet> python tcp_server.py
TCP服务器已启动, 监听端口6121, 等待客户端连接...
[新连接] 客户端('192.168.31.197', 52780)已连接
[响应完成] 已处理客户端('192.168.31.197', 52780)的请求
[连接关闭] 客户端('192.168.31.197', 52780)的连接已断开
```

图 2.6: tcp_server_diff

```
(llm) PS D:\VSCodeProject\homework\ComputerNet> python tcp_client.py
RTT: 0.008181700017303228 s
响应报文:
HTTP/1.1 200 OK
Connection: close
Date: Sat, 27 Dec 2025 06:14:03 GMT
Server: Apache/1.3.0 (Windows)
Last-Modified: Mon, 28 Nov 2022 00:00:00 GMT
Content-Length: 20
Content-Type: text/html; charset=utf-8

request successfully
```

图 2.7: tcp_client_diff

4 问题与体会

报告中所示代码对老师提供的代码进行了优化, 修复了 HTTP 报文中的部分不规范的地方 (虽然不影响代码运行), 另外实现了服务端的多线程处理功能。

本次实验让我深刻认识了 TCP 面向连接的可靠传输特性, 以及 HTTP 1.1 的报文格式。从解析请求、构造 200/404 响应到实现多线程并发, 我掌握了 TCP 套接字编程, 还明白了 Web 服务器处理请求的核心流程, 体会到了协议规范与实际应用的衔接要点。

实验 3 SMTP 客户端实现

1 实验目的

进一步理解和掌握基于 Python 进行 TCP 套接字编程的知识, 理解 SMTP 报文格式, 能基于 Python 编写一个简单的 SMTP 客户端程序。

2 实验条件

装有 python 环境的电脑两台;
局域网环境。

3 实验内容及原理

3.1 实验原理

简单邮件传输协议 (Simple Mail Transfer Protocol, SMTP) 是实现电子邮件收发的主要应用层协议, 它基于 TCP 提供的可靠数据传输连接, 从发送方的邮件服务器向接收方的邮件服务器发送邮件。注意, 虽然一般情况下邮件总是从发送方的邮件服务器中发出, 但是工作在发送方邮件服务器上的发送程序是一个 SMTP 客户端, 因此一个完整的 SMTP 程序总有两个部分参与工作: 运行在发送方邮件服务器的 SMTP 客户端和运行在接收方邮件服务器的 SMTP 服务器。它限制所有邮件报文的主体部分只能采用简单的 7 比特 ASCII 码表示, 所以在用 SMTP 传送邮件之前需要将二进制数据编码为 ASCII 码, 在传输后再进行解码还原为二进制数据。

3.2 实验步骤

完成一个简单电子邮件客户端程序, 并通过向不同的账号发送电子邮件来测试程序。

3.3 代码说明

基于 SMTP 协议，通过 QQ 邮箱的 SMTP 服务器发送纯文本邮件到指定收件人邮箱，处理邮件发送过程中的异常情况。

1. 配置 SMTP 服务参数：指定 QQ 邮箱的 SMTP 服务器地址 (smtp.qq.com)、发送方邮箱、SMTP 授权码、收件人邮箱。
2. 构造邮件内容：使用 email.mime.text.MIMEText 创建纯文本邮件对象，指定邮件内容、格式、编码，通过 Header 设置邮件的发件人、收件人、邮件主题。
3. 建立 SMTP 连接并登录：使用 smtplib.SMTP_SSL 创建加密 SSL 连接，通过 login() 方法传入发送方邮箱和授权码，完成身份验证。
4. 发送邮件并处理异常：通过 sendmail() 方法发送邮件，使用 try-except 捕获 smtplib.SMTPException 异常，处理邮件发送失败的情况，打印对应的错误信息或发送成功提示。

```
1 import smtplib
2 from email.mime.text import MIMEText
3 from email.header import Header
4
5 # 第三方SMTP服务
6 mail_host = "smtp.qq.com" # 设置服务器
7 # 自己的邮箱，通过QQ邮箱设置获取口令
8 mail_user = "2723733688@qq.com" # 发送方: QQ邮箱
9 mail_pass = " " # 获取授权码
10 sender = "2723733688@qq.com" # 发送方qq邮箱
11 receivers = ["2311451@mail.nankai.edu.cn"] # 接受者的邮箱,另一个邮箱
12
13 # class email.mime.text.MIMEText(_text[, _subtype[, _charset]]) 用于创建主要类型文本的MIME对象, _text是有效负载的字符串, _subtype
14 # 是次要类型, 默认为plain, _charset是文本的字符集
15 message = MIMEText('I love computer networks', 'plain', 'utf-8')
16 message['From'] = Header(f"lizonghang <{sender}>") # 显示名和发件人邮箱
17 message['To'] = Header(f"<{receivers[0]}>", "utf-8") # 收件人邮箱
18 message['Subject'] = Header("邮件测试") # 邮件主题
19
20 try:
21     smtpObj = smtplib.SMTP_SSL(mail_host, 465)
22     smtpObj.login(mail_user, mail_pass) # 会返回(状态码, "字符串解释")元组信息
23     smtpObj.sendmail(sender, receivers, message.as_string())
24     print("邮件发送成功")
25 except smtplib.SMTPException as e:
26     print(e)
27     print("Error: 无法发送邮件")
```

图 3.1: smtp

3.4 实验结果

通过 QQ 邮箱分别向两个不同的邮箱发送邮件，可以看到均能发送成功。

```
(llm) PS D:\VSCodeProject\homework\ComputerNet> python smtp.py
邮件发送成功
(llm) PS D:\VSCodeProject\homework\ComputerNet> python smtp.py
邮件发送成功
```

图 3.2: smtp_console

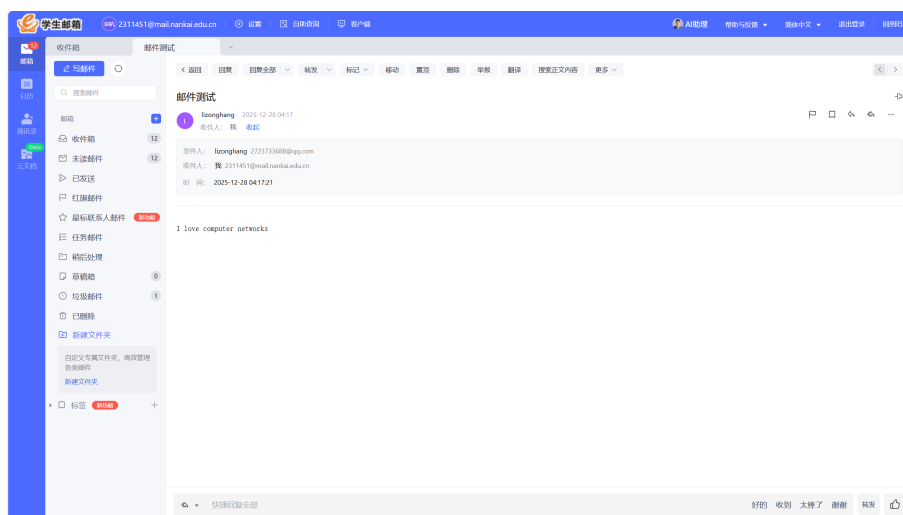


图 3.3: smtp_edu

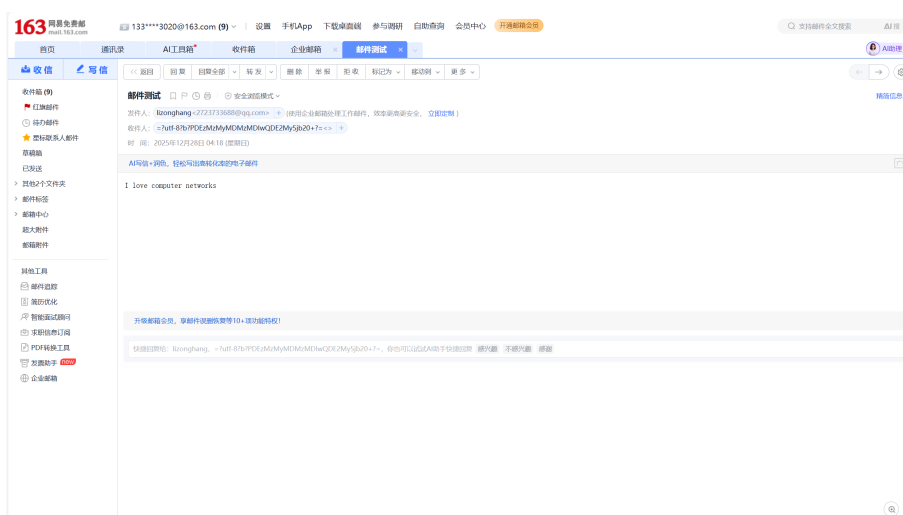


图 3.4: smtp_163

4 问题与体会

通过此次 SMTP 客户端实验，我了解了基于 TCP 的应用层协议交互逻辑，学习了 smtp-lib 库的使用。配置邮箱授权码、建立 SSL 连接的过程，让我了解了邮件服务的安全机制和 SMTP 命令与响应码的通信规范，深化了对“TCP 可靠传输支撑上层协议”的理解。